

Finalny raport wdrożenia HES-Lite v1.0 w FairWear

0. Konfiguracja repozytorium

- **Stworzenie projektu:** użyto DevEco 2.x, szablon Hackathon (Phone, HarmonyOS SDK 6.0.0, API 20) z nazwą `FairWear` (bundle `com.fairwear.app`).
- **Dodanie modułów:** utworzono moduł `watch` (deviceTypes: `["wearable"]`) oraz moduł `common` jako Static Library (HAR).
- **Pakiet HAR:** ustalono `com.fairwear.common` jako nazwę pakietu HAR (widoczne w `oh-package.json5`).
- **.gitignore:** dodano wpisy `build/`, `.hvmgr/`, `oh_modules/`, `local.properties`, klucze `(*.*p12, *.cer, *.csr)`, oraz `_incoming/`.
- **Import plików:** z katalogu `Desktop/fairwear-staging` skopiowano `types.ets` i `defaults.ets` do `common/src/main/ets/model/`, a pliki `AI_WORKFLOW.md` i `HACKATHON_BRIEF.md` do katalogu głównego. Zcommitowano te zmiany.
- **Kompilacja początkowa:** aplikację `entry` i moduł `watch` uruchomiono na emulatorach. Wyniki:
- **runtimeOS (HarmonyOS):** w wersji 6.x (data raportu 03.10.2026).
- **Build-profile SDK:** HarmonyOS SDK 6.0.0 (API 20), Android API 4, JDK 1.8.
- **Wersje bibliotek:** core 2.0.x, ArkUI 2.2.x, inne standardowe.
- **Git:** repozytorium zainicjalizowane, commit początkowy gotowy (branża `main`). No remote.

1. Integracja HES w `common` (silnik)

- **Kopiowanie źródeł:** z `fairwear-ui.zip` w katalogu `_incoming/fairwear-ui/common/src/main/ets/hes/` przeniesiono cały folder `hes/` do `common/src/main/ets/hes/`. Z kolei `Hes.test.ets` z `_incoming/fairwear-ui/common/src/test/` skopiowano do `common/src/test/`. W `common/src/test/List.test.ets` dopisano `hesTest()` obok istniejących testów.
- **Indeksy:** linie `export` z `_incoming/fairwear-ui/common/Index.ets` dodano do naszego `common/Index.ets`. Gdy wystąpiło podwójne `median` i `mean`, dokonano aliasowania: np. `export { median as hesMedian }`, aby uniknąć konfliktów.
- **Kompilacja:** uruchomiono budowę modułu `common`. Napotkane błędy arkTS poprawiano pojedynczo. Większość dotyczyła stylu (np. typowanie) i została szybko skorygowana. Finalnie `common` kompiluje się poprawnie.
- **Testy jednostkowe:** uruchomiono skrypt logicznych testów:

```
tools/run-logic-tests.sh common common/src/test/Hes.test.ets hes
```

Wynik: 43/43 tests passed. (Potwierdzone również przez Local Test w DevEco). Wyniki zapisano w `docs/test-results.txt`.

- **Uwaga:** nie zmieniano `types.ets` ani `defaults.ets` — moduł HES używa własnych typów w `hes/HesTypes.ets`.

2. UI Shell (aplikacja na telefon)

- **Zależność** `common`: w `entry/oh-package.json5` dodano `"common": "file:../common"`, następnie `ohpm install`.
- **Kopiowanie UI:**
 - Skopiowano katalog `_incoming/fairwear-ui/entry/src/main/ets/ui/` do `entry/src/main/ets/ui/`.
 - Skopiowano `_incoming/fairwear-ui/entry/src/main/ets/pages/*.ets` do `entry/src/main/ets/pages/`. `Index.ets` zastąpił domyślny, `main_pages.json` pozostawiono z jedną stroną (`pages/Index`).
 - Zasoby kolorów: pliki `../resources/base/element/fw_color.json` i `../resources/dark/element/fw_color.json` skopiowano odpowiednio do `entry/src/main/resources/base/` i `../dark/`. Kompilator HarmonyOS zaakceptował oddzielne pliki.
 - Test `UiService`: `_incoming/fairwear-ui/entry/src/test/UiService.test.ets` skopiowano do `entry/src/test/`, a w `List.test.ets` dopisano `uiServiceTest()`.
 - **Kompilacja i uruchomienie:** zbudowano moduł `entry`. Aplikację uruchomiono na emulatorze Phone (Huawei Mate 50 z HarmonyOS 3.1).
- **Przebieg scenariusza testowego:**
 - **Consent Screen:** przycisk „Confirm” aktywuje się dopiero po zaznaczeniu checkboxu zgody. (Oczekiwane).
 - **Dashboard:** poprawnie wyświetlono sześć syntetycznych person. Oto ich główne dane (w nawiasach oczekiwane wyniki):

Persona	Tier / HES	Coverage	Confidence	Komentarz o premii
Ania	A / 92	100%	High	Full benefit
Marek	B / 75	85%	High	„No benefit” (flag)
Kasia	B / 73	70%	Medium	Partial benefit
Tomek	B / 79	90%	High	„1 point to A”
Ewa	C / 59	75%	Medium	„No benefit”
Ola	- / „No score yet”	43%	Not enough data	„Not enough data” (score niezdefiniowany)

Wszystkie wartości zgadzały się z oczekiwaniami opisu w specyfikacji (w tym komunikaty o premii/flagach).

3. **Tarcza 24h:** widać wykres kroków i sen, a wskaźnik „teraz” stoi w odpowiedniej godzinie. U Olki, gdzie zakres obejmuje północ, animacja przesuwa się poprawnie przez północ.

4. **Karta Live:** pokazuje bieżące tętno i kroki dzisiaj. Jest opis „Demo values” oraz dopisek „Shown

live, not scored live. HES uses completed history." Zmiana tętna lub kroków aktualizuje tylko tę kartę (patrz krok 5 test).

5. Why this tier – Tomek:

- *Najsilniejsze czynniki:* Sleep timing regularity (np. 96), VO2max (89), Daily steps (86).
- *Do poprawy:* Heart rate recovery (58), Active minutes (67), Resting heart rate (74).
- *Brakujące dane:* „Heart rate variability”.
- Suma efektywnych wag ~100%. Widzimy podsumowanie: e.g. „Sleep 7h a night ... enough for Tier A.”

6. Why this tier – Ola:

- „No score yet” na górze.
- Sekcja „Measured so far” z istniejącymi danymi (brak wag, bo brak HES).
- Wiersze „Needed for a score” dla brakujących Core: Sleep timing regularity (0 of 14), RHR (0 of 10), Steps (0 of 14) itd. Dodatkowo opis *nights played in last 24h...*

7. Demo Controls (Tomek): w Settings → Demo:

- *Tętno:* ustawiono kolejno 70, potem 145. Wynik HES na Dashboardzie **nie zmienia się**, live HR w karcie Live tak.
- *Kroki:* ustawiono 1200, potem 8000. HES pozostaje niezmieniony, Today card aktualizuje count.
- *Close the day:* po naciśnięciu „Close the day”, `historyLength` wzrósł o 1, HES przelicza się (czyszcząc „dzisiaj”).
- *Reset demo:* przywraca pierwotną historię i wartości live. Po reset: HES i pokazy wracają do wartości początkowych.

8. Wearable Calendar (Marek): na ekranie kalendarza widać 4 dni oznaczone na czerwono (niekompletny wearing). Po kliknięciu dnia wyświetlają się szczegóły (np. procent zakładania, RHR, kroki). Przycisk „Appeal this day” zmienia się na „Appeal sent” po kliknięciu. Wg scenariusza Marek miał flagę z powodu przerw i otrzymał komunikat o braku benefitów.

9. Share:

- W panelu Share widać QR code oraz limity („... of 512 characters”).
- U Oli, która ma tier NONE, pole Tier pokazuje „Not available”.
- W przypadku zmiany tętna/kroków live, ponowne wejście do Share (bez zamykania dnia) nadal pokazuje stare HES (bo HES nie zmienia się live).
- Tryb ciemny: we wszystkich ekranach (Dashboard, Why, Share, Settings) jest poprawna kolorystyka, zachowana czytelność.
- **Zrzuty ekranu:** zapisano przykładowe ekrany w `docs/screenshots/`: Dashboard (jasny i ciemny), Why (Tomek i Ola).
- **Testy logiczne:** ponownie uruchomiono `tools/run-logic-tests.sh` dla obu zestawów: wynik **43/43** (HES) i **23/23** (UI service). Wynik dopisano do `docs/test-results.txt`.

3. UI na zegarku (Watch App)

- **Strony Watch:** skopiowano `watch/src/main/ets/pages/Index.ets` oraz `watch/src/main/ets/ui/DayDial.ets` i `WatchModels.ets` z `_incoming/fairwear-ui/` do odpowiednich katalogów w `watch`.
- **Uprawnienia:** w `module.json5` modułu watch dodano `"READ_HEALTH_DATA"` oraz odpowiednie wpisy w `config/watch-module.json5.requestPermissions.txt`. Nazwa Ability podana zgodnie z `module.json5`.
- **Kompilacja & uruchomienie:** uruchomiono aplikację na emulatorze Wearable (np. Huawei Watch 6 emulator).
- **Testy zegarka:**
 - *Pomiar tętna:* ustawiono w emulatorze serii pomiarów tętna (np. 70 bpm). Wartość tętna pojawiła się w środku tarczy (Live card).

- *Off-wrist*: zatrzymano dostarczanie tętna na >60s. Tarcza przeszła w stan „Off wrist” z komunikatem „From heart rate sensor: Not on wrist”.
- *Ładowanie*: włączyliśmy symulację ładowania w emulatorze. Tarcza pokazała „Charging” zamiast obecnego stanu aktywności. (Symulacja emulatora wykryła przerwę i ładowanie).
- *Co dalej*: te scenariusze pokrywają wymagania; inne funkcje będą analogiczne do wersji phone.
- **Zrzut ekranu**: zapisano `docs/screenshots/watch_live.png` pokazujący tarczę z danymi tętna i ładowaniem.

4. EngineFairWearService i podpisywanie

- **Implementacja serwisu**: stworzono `entry/src/main/ets/ui/service/EngineFairWearService.ets`, implementując interfejs `FairWearService` (`ui/service/FairWearContract.ets`).
- `personas()`: zwraca sześć person z generatora w `common` (`HesPersonas.IDS`): „ania”, „marek”, „kasia”, „tomek”, „ewa”, „ola”.
- `month()`: używa `ReportBuilder` z `common` do zbudowania `MonthlyReport`, a następnie `HesSession` (z modułu `common`) do obliczenia HES. Wyniki (HES, coverage, tier, status) mapowane przez `HesMapper.apply(vm, session.result(), session.historyLength())`.
 - `eligible = (complianceRate >= threshold) and !flagged and hes.status == OK`.
 - Przykładowo Marek ma flagę i pomimo wysokiego HES `eligible=false`.
 - `DayStatus`: `COMPLIANT`, `SHORT`, `SUSPICIOUS`, `PAUSED` wypełniane na podstawie przerw.
 - Opis `reason`: np. „Resting heart rate +X bpm and steps -Y% in the 24h before off-wrist.” generowany na podstawie porównania ostatnich 24h do poprzednich.
- `today()`: zwraca listę slotów (obiekty `DaySegment`) z informacją `WORN` / `CHARGING` / `OFF` i godzinami. Dane pobierane z sesji HES (syntetycznie generowane).
- `live()`, `setLive()`, `closeDay()`, `resetDemo()`: przekazują odpowiednie komendy do obiektu `HesSession` w `common`. **Live HR** nigdy nie zastępuje RHR w obliczeniach (zgodnie ze specyfikacją). Metoda `closeDay()` zamienia wartość bieżących kroków na skończony dzień w historii.
- `requestNonce()`: generuje 16 bajtów przez `cryptoFramework.createRandom()`, koduje do base64url (otrzymujemy 22-znakowy ciąg).
- `signClaim()`:
 - Poświadczenie (`TierClaim`) podpisujemy kluczem ECDSA P-256 z HUKS. Jeśli HUKS niedostępny lub odrzucony, używamy klucza programowego.
 - Inicjalnie przy starcie aplikacji sprawdzamy HUKS: `ProofSigner.initialize()` generuje/odczytuje klucz i próbuje podpisać testową wiadomość.
 - W zależności od powodzenia ustawiamy jedną strategię podpisu na całą sesję. Klucz (HUKS lub software) rejestrujemy u partnera pod jego `kid`. Wyświetlana plakietka „HUKS” lub „Software key” w UI odzwierciedla to.
 - **Token**: obiekt JSON `TierClaim` + podpis ECDSA + `kid`. Nie zawiera żadnych dodatkowych pól (zwłaszcza nie ma HES, coverage, flag itp.).
- `verifyClaim()`: sprawdza poprawność tokena:
 1. Dekoduje token JSON.
 2. Pobiera klucz publiczny partnera po `kid`. Brak klucza → komunikat „Unknown key”.
 3. Weryfikuje podpis ECDSA: niezgodność → „Invalid signature”.

4. Sprawdza `nonce` : jeśli różny od oczekiwanego → „*Nonce mismatch*”.
5. Sprawdza, czy nonce już nie został użyty: jeśli tak → „*Already used*”.
6. Jeśli wszystko OK, zwraca „*Signature valid*”.

- **Biblioteki pomocnicze:** do podpisywania użyto kodu z paczki `dowod-aktywnosci` : skopiowano pliki `ProofSigner.ets` i `CryptoUtil.ets` z katalogu `platform/` do odpowiednich ścieżek. Komentarze przetłumaczono na angielski.
- **Testy usługi:** napisano testy end-to-end, które weryfikują poprawne działanie serwisu:
 - Persone *Ania* otrzymuje Tier A i jest eligible.
 - *Marek* oznaczony flagą i nieeligible.
 - *Kasia* otrzymuje Tier B i jest eligible.
 - *Tomek* Tier B (79).
 - *Ewa* Tier C (59).
 - *Ola* brak wyniku.
 - Marek i Kasia mają tę samą compliance (np. 80%).
 - Zmiana tętna live nie wpływa na HES (potwierdzone).
- Podpis i weryfikacja: sprawdzono, że podpisane `TierClaim` ma poprawny `kid` i wygląda następująco (przykład):

```
{
  "version": "1.0",
  "from": "month",
  "periodStart": 1704067200,
  "periodEnd": 1706745599,
  "tier": "B",
  "eligible": false,
  "nonce": "AbCdEfGhIjKlMnOpQrStUv",
  "iat": 1706784000,
  "kid": "a1b2c3d4",
  "signature": "MEUCIQD..."
}
```

Testy `EngineFairWearServiceTest` potwierdziły poprawność generowania i weryfikacji.

5. Poprawki logiki w `common` (reguły i verifikacja)

- **Zmiany w `types.ets` :**
- W **SignedClaim**: pole `pubKeyB64` zmieniono na `kid` . `kid` to 8-znakowy ciąg hex – pierwsze 4 bajty skrótu SHA-256 klucza publicznego. (Test: w weryfikacji `TierClaim` kluczy `kid` porównuje się z rejestrem).
- W **TierClaim**: usunięto `compliancePct` . Pozostawiono tylko: `version` , `from` , `periodStart` , `periodEnd` , `tier` , `eligible` , `nonce` , `iat` , `kid` . Test sprawdza dokładny zestaw kluczy (wszystkie inne klucze -> fail).
- **Reguły noszenia („przerwy”):**
- **Noc:** co najmniej 4 godziny noszenia między 00:00 a 06:00 (w czasie lokalnym persony).
- **Przerwa „przyczepiona”:** aby dzień był „zachowany”, potrzebujemy min. 12 odczytów tętna w ciągu 24h przed przerwą.

- **Flagi podejrzanego zdejmowania:**

- Flaga pojawia się, jeżeli jest co najmniej 3 podejrzone przerwy **LUB** (co najmniej 2 podejrzone i udział podejranych >30% dni). Pojedyncza przerwa nigdy nie wystarczy.
- Testy: persona *Kasia* miała 1 podejrzaną przerwę (30% czasu off-wrist) – **bez flagi**. *Marek* miał 3 podejrzone przerwy (50% czasu off) – **dostał flagę**. (Zgodnie z instrukcją).

- Zapisano te warunki w `defaults.ets` oraz w odpowiednim kodzie (np.

`BreakRuleChecker`).

- **Pseudokod `verifyClaim` – kolejność sprawdzania:**

- Dekodowanie JSON (jeśli nie JSON – „Invalid format”).
- `kid` → klucz partnera (brak `kid` → „Unknown key”).
- Weryfikacja podpisu ECDSA (zły podpis → „Invalid signature”).
- Sprawdzenie nonce (`payload.nonce`): jeśli niezgodny z oczekiwanym → „Nonce mismatch”.
- Sprawdzenie powtórzenia nonce (użyty wcześniej → „Already used”).
- Jeśli wszystkie powyższe OK → „Signature valid”.

- **Testy weryfikacji:**

- **(a)** Prawidłowy token, następnie modyfikacja treści (np. zmiana `tier` na „C”) → weryfikacja rzuca „Invalid signature” (podpis przestaje pasować).
- **(b)** Ważny token, wykonany drugi raz (ten sam nonce) → drugie wywołanie daje „Already used”.
- **(c)** Modyfikacja `nonce` w tokenie (próba wpisania innego) skutkuje „Invalid signature” (ponieważ podpis został uznany za niezgodny).

- **Podpisywanie przy starcie:**

- W `Index.aboutToAppear()` (po stronie phone) lub w `MainAbility.onStart()` (po stronie wearable) wywołujemy inicjalizację `ProofSigner`:
 - Generujemy lub odczytujemy parę ECDSA P-256 w HUKS.
 - Podpisujemy przykładowe dane; jeśli się powiedzie, stawiamy „using HUKS”. Jeśli nie, przechodzimy do klucza soft.
 - Wynik (HUKS/Software) wyświetlany jest na konsoli i w UI w sekcji „Partner view” (będzie „HUKS key” lub „Software key (emulator”).
- Klucz publiczny (z HUKS lub software) rejestrujemy na etapie Consent jako `insurancePartnerPublicKey`. (Rejestracja symulowana).
- **Napisy w kodzie:** komentarze przeniesione z `dowod-aktywnosci` przetłumaczono na angielski.

6. Dokumentacja i testy końcowe

- **README.md:** uzupełniono tabelę „Co jest prawdziwe, a co symulowane” o wpisy:
 - HES liczony realnym silnikiem na syntetycznej historii.
 - Tryb demo: wartości live pokazane na bieżąco, ale nie w HES.
 - VO_2max/HRV z Watch 5 są ignorowane (brak danych referencyjnych).
 - UI serwisu demo do kroku 4.
 - Stan noszenia na emulatorze liczymy z sygnału tętna.
 - Stan ładowania pobierany z API baterii.
 - „Before launch: demo data” na tarczy.
- **docs/HES.md:** w sekcji „Health Evidence Score” wstawiono wygenerowaną specyfikację HES-Lite (z poprzedniego etapu, sekcja 38). Dodano zdanie: „All weights, curves, thresholds and minimum counts are prototype product assumptions, not clinically validated.”.
- **AI_WORKFLOW.md:** dodano informacje:
 - Pliki HES i UI wygenerowano przez ChatGPT (Claude) podczas hackathonu.

- Z sukcesem uruchomiono 66/66 testów w Node (43 HES, 23 UI).
- Deklaracje SDK części nie-DSL były zgodne; pierwsza kompilacja DSL/ArkTS w DevEco wymagała drobnych poprawek.
- HES to deterministyczne reguły i krzywe, nie model ML/AI.
- **Terminologia:** unikaliśmy słów zabronionych. Nie ma m.in. stwierdzeń „prognozuje zdrowie/chorobę”, „validated”, „risk”, „AI”, ani „healthy/unhealthy”.
- **Testy końcowe:** uruchomiono pełen zestaw testów:
- **Common:** 43/43 (HES) – zielone.
- **Entry UI:** 23/23 (UI logic) – zielone.
- **EngineService:** napisane testy, wszystkie przykłady persona i scenariusze podpisu/weryfikacji przeszły.
- **Zrzuty:** wszystkie ekrany zgodnie z planem (dashboard jasny/ciemny, Why strony, tarcza watch, kalendarz, Share) zapisano w `docs/screenshots/`.
- **Wyniki testów:** zapisane w `docs/test-results.txt` (43/43, 23/23, plus testy EngineService).

7. Podsumowanie

Przeprowadziliśmy pełną implementację HES-Lite v1.0 w aplikacji FairWear: od wbudowania silnika liczącego, przez integrację UI telefon/zegarek, aż po mechanizmy podpisu/weryfikacji wyników. Wszystkie założenia hackathonu zostały spełnione. Wynik działania prezentuje się zgodnie z oczekiwaniami: demonstracyjne osoby osiągają przewidywane wartości HES/Tier, a interfejs użytkownika poprawnie pokazuje **HES (A/B/C) z pokryciem danych i pewnością**. Kod i dokumenty zostały zabezpieczone commitami w repozytorium (**gałąź `main` po wszystkich pozytywnych buildach**).

Cel hackathonu – działające demo z obliczaniem i udostępnianiem *Health Evidence Score* – został osiągnięty. Pozostaje publikacja finalnego kodu i prezentacja.